

F1R3COMB

A compiler from kernel rho to the name-free rho combinators,
and a machine that runs the result

Specification v0.4

F1R3FLY.io

23 September 2026

Abstract

[3] has been revised again. The revision answers every item v0.3 raised, and answers three of them by promoting a remark to a lemma with a stated side condition — which is the outcome a specification wants and also the outcome that gives it the most work, since a side condition is something an implementation must be built to satisfy rather than something it may assume.

The substance: [3, Rem. 6.10], the un-proved harmless claim on which phase two rested, is now the single-address-input discipline [3, Def. 6.10] and the permutation lemma [3, Lem. 6.11]. Phase-two freshness is now [3, Lem. 6.14], by a size argument rather than the shape argument the constructor family closed off, with a second clause that is an obligation on the translation. The family has a member for the storage atom [3, Def. 3.8], and its finiteness is statable [3, Rem. 6.15]. Per-node gadget channels and one spare leaf per nested unit are [3, Rem. 6.13]. The atom count is weighted [3, Lem. 9.2].

Three things follow for v0.4. The discipline is not free: ERECT as [3, Def. 6.8] states it *violates* [3, Def. 6.10] for every atom mentioning two or more bound names, which is most of them, so the scheme of [3, Rem. 6.12] is effectively mandatory, and satisfying it requires a two-stage erection this specification sets out (Section 5.4). Freshness by size is easy to check and Section 5.5 makes it a deploy-time test, while noting that [3, Def. 6.8] still states no freshness condition on the static channels themselves, which is the last place an encoded context could reach the apparatus. And v0.3's claim that phase two is presentation-A-only was wrong in the interesting direction: [3, Rem. 3.9] shows B builds the storage constructor *more* directly than A, so the point is withdrawn and Decision 2.1 now rests on its other three reasons, which are undisturbed. Key words MUST, MUST NOT, SHOULD, MAY are used in the sense of RFC 2119.

Contents

1	Purpose and scope	2
1.1	The gap	2
1.2	What exists today	3
1.3	Nomenclature	3
1.4	Goals and non-goals	3
2	Which presentation	4
3	The source language	5
4	The target calculus	5
4.1	Formation, equations, rules	5
4.2	The constructor family	6
4.3	Representation, encoding, hashing	7

5	The two-phase translation	8
5.1	The bug, and why the specification had it too	8
5.2	Phase one	8
5.3	Phase two	9
5.4	The permutation discipline	9
5.5	Freshness	11
6	Canonicity	11
6.1	What survives, and what it now costs	11
7	Architecture	13
8	The lowering	13
8.1	Phases	13
8.2	Occurrence analysis	14
8.3	Constructor chains for (o5)	14
8.4	Shortcuts	15
8.5	Worked examples	15
9	The machine	15
9.1	The instantiation	15
9.2	Addresses	16
10	The lattice audit	16
11	Correctness obligations and testing	17
12	Errors and diagnostics	18
13	Performance and portability	18
14	What the implementation asks of the papers	19
15	Open questions	20
16	Delivery	20
17	Changes from v0.3	21

1 Purpose and scope

1.1 The gap

[3] defines a translation from the rho calculus into the repaired combinators, in two phases, and proves it correct as full abstraction relative to encoded contexts. [4] gives the spatial-behavioural logic over the same calculus. Neither has been run. The properties a paper translation may leave up to bisimilarity — which address, which static channel, which representative of a constructed name, which presentation — are properties an implementation may not leave open, because they decide whether two independently compiled terms can speak to one another.

The `campf1r3` workspace is a complete executive for kernel rho: `kindling-parse` and `kindling-norm` take text to a canonical normal form, `campf1r3-core` is the store, `b3ll0ws` interprets the normal form, and `f1r3x` drives them. It knows nothing of combinators. `F1R3COMB` is the missing arrow, plus the smallest machine that can run its output.

1.2 What exists today

Artefact	What it gives this project
<code>rho-combinators-draft3</code>	The target as an operational theory (§3.2–3.3), the two presentations and the argument between them (§3.3, §4), the constructor family (Def. 3.8), erection at a computed name (Rem. 3.11), the positional-naming bug (§6.1), phase one (§6.2), phase two (§6.3), the permutation discipline (§6.4, Def. 6.10, Lem. 6.11), phase-two freshness (§6.5, Lem. 6.14), the correctness lemmas (§7), the lattice (§9)
<code>rhocomb-logic</code> (draft 2 rev. 1)	The logic; the presentation decision (Dec. 2.3), what B costs it (Prop. 2.4, Rem. 2.5), drop elimination (Lem. 2.8), $\equiv_{\mathcal{N}}$ decided (Prop. 2.10), the weighted atom count (Def. 6.6, Prop. 9.2 there), occupancy in linear time
<code>kindling-spec</code>	$K0$ = pure rho; the AST, the normal form, its encoding and hashing (Def. 2.6, Req. 2.7, Prop. 6.9)
<code>campf1r3-spec</code>	The store: D, W, I , produce/consume/install, joins, profiles, replay
<code>bytecode-or-normal-form</code>	Why code must never be a channel key unless the compiler is canonical
<code>campf1r3</code> (the repository)	The eight crates, no external dependencies

1.3 Nomenclature

Term	Meaning here
<i>source</i>	$K0$ of [5], the rho calculus of [3, §2] modulo Obligation 3.2
<i>target</i> , RC	The theory of [3, §3]: two sorts, nine atom shapes plus the constructor family, $\mathbf{0}$, par, quote, drop, no binders
RC^ν	The target extended with $(\mathbf{new} z)P$; the codomain of phase one and the compiler’s intermediate representation
<i>presentation</i> A, B	[3, §3.3]; A adds $*@p = p$ and sorts message payloads as names
<i>router</i>	d, k, fw, b_l, b_r, s
<i>constructor</i>	$\mathbf{cons}_A$ for an atom shape A , per [3, Def. 3.8]
<i>address</i>	The name an instance receives at run time and allocates beneath
<i>leaf</i> , $\sigma \cdot w$	An allocated name, at path w beneath address σ
<i>static channel</i>	A name appearing in the phase-two image itself, shared by every instance
<i>unit</i>	One quotation’s worth of phase-one output: the top-level term, or the body of one input, phase-two-compiled on its own
<i>instance</i>	One run-time copy of a unit, at one address
<i>link</i>	The atom emitted for one occurrence, per Table 1

1.4 Goals and non-goals

1. A total, canonical, terminating compiler implementing [3, Def. 6.3 and 6.8], with every degree of freedom in them closed.
2. A target representation with a canonical encoding and content hash, so that $\equiv_{\mathcal{N}}$ is a 32-byte comparison, and an intermediate representation carrying the phase-one binder.
3. A machine that runs target terms by instantiating CAMPF1R3.
4. Differential testing against B3LL0WS, including the two-instance interleaving that [3, §6.1] and [3, Rem. 6.10] make the crux of correctness.

5. A term type and an occupancy report [4] can build on.
6. No external dependencies, `forbid(unsafe_code)`, WebAssembly-clean.

Out of scope: the logic itself; K1 and K2; the π route; decompilation; cost accounting; the matrix encoding of the reduction graph; optimisation beyond [3, Rem. 6.4].

2 Which presentation

[3, §3.3] gives two sound presentations and [3, Rem. 3.2] shows they cannot be mixed piecemeal. A compiler cannot hold both: the term type, the datum type, the tag table and the decision procedure for $\equiv_{\mathcal{N}}$ all differ.

	Presentation A	Presentation B
m payload sort	N	P
eval	$e(a) \mid m(a, v) \longrightarrow *v$	$e(a) \mid m(a, p) \longrightarrow p$
$*-$ in a term	eliminable; absent from normal forms ([4, Lem. 2.8])	irreducible; needed
$@-$ injective	yes, on normal forms ([4, Prop. 2.10])	no ([4, Prop. 2.4(ii)])
$\equiv_{\mathcal{N}}$ decided by	byte equality of the encoding	a separate normalisation, strictly coarser
cons_q	$@q(v_1, *v_2)$: available, but its correctness is the equation	$q(@p_1, p_2)$: direct, the sorts already fit ([3, Rem. 3.9])
Logic support	full	needs a receptivity primitive ([4, Rem. 2.5])

Decision 2.1 (F1R3COMB implements presentation A). Unchanged from v0.2, and now over-terminated. The reasons of v0.2 stand: the logic is a logic for A and under B cannot distinguish a store from a message; $@-$ is injective only under A and CAMPF1R3 keys channels by the encoding; [3, Def. 3.3] writes the translation in A’s notation; the representation is smaller; and `kindling-norm` already implements A on the source side.

Remark 2.2 (v0.3’s fourth reason is withdrawn). v0.3 argued that phase two must erect the gate’s storage atom and that only A can build the constructor for it, making phase two A-only. [3, Def. 3.8] and [3, Rem. 3.9] settle it the other way. The member exists in both: under B it is $\text{cons}_q(a, b, c) \mid m(a, p_1) \mid m(b, p_2) \longrightarrow m(c, q(@p_1, p_2))$, where the sorts already fit; under A it must be written $@q(v_1, *v_2)$, coercing a name back to a process, which denotes the intended atom only because $*@p = p$. So this is a third place the presentations come apart, and it leans mildly towards B. It does not move the decision: within A the coercion is sound by the equation A asserts, so the cost is presentational rather than real, and the three reasons above are untouched. It is recorded because v0.3 claimed the opposite, and a reader of both documents is entitled to know which is right.

Remark 2.3 (The §4 argument of [3], still outstanding). v0.2 §2.3 answered [3, §4]’s reading of the F1R3FLY normaliser as evidence for B: sorting a parallel composition is the monoid laws, common to both; de Bruijn indexing applies to source binders and the combinators have none; and the one presentation-specific rewrite the code performs, $*@P \rightarrow P$, is A’s equation. That section is unchanged in the revision, so the point stands as written.

Requirement 2.4 (Presentation is a compile-time parameter). `presentation-b` MUST be a cargo feature, off by default, coupling the equation, the payload sort and the **eval** rule in one switch ([3, Rem. 3.2]). It MUST be recorded in every artefact header and a deploy MUST diagnose

a mismatch. Both presentations admit the whole translation, including the erection of storage atoms (Remark 2.2), so the seam has no structural exclusion and **MUST** be exercised across the whole corpus (Obligation 11.8).

3 The source language

[3, Def. 2.1]	K0	k1ndling-norm Node
0	Nil	Nil
$P \mid Q$	$P \mid Q$	Par (Vec<Norm>), sorted, no Nil
$x!(P)$	$x!(P)$	Send {chan, persistent:false, args:[P]}
$\text{for}(y \leftarrow x)P$	$\text{for}(y \leftarrow x)\{P\}$	Receive {binds:[b], body}
$*x$	$*x$	Eval (Name)
$@P$	$@P$	Name::Quote (Norm)

Requirement 3.1 (Source level and closedness). **F1R3COMB** **MUST** accept exactly K0, invoking **k1ndling_parse** with **Level::K0**, and **MUST** reject every K1 and K2 form with a distinct diagnostic. The term **MUST** be closed: no **FreeVar**, no **FreeName**. **Name::Unforgeable** **MUST** be rejected, because the target has no name former for it; grounding **MUST** be by quotation.

Obligation 3.2 (The source presentations agree on guarded-drop terms). [3, §2] states the source congruence without $*@P \equiv P$; [5, Def. 2.6] asserts it and **Norm::eval** applies it eagerly. For P guarded-drop ([3, Def. 2.5]) no subterm $*@Q$ occurs in P or any reduct ([3, Lem. 2.6]), so the rewrite never fires and the congruences coincide. This **MUST** be proved; it is what licenses compiling a **K1NDLING** normal form against [3, §2].

Requirement 3.3 (Guarded-drop check). The compiler **MUST** verify the guarded-drop property on the *tree*, before normalisation, since **Norm::eval** has already collapsed $*@Q$ by the time the normal form exists. Under Decision 2.1 violations are warnings and compilation **MAY** proceed; under **presentation-b** they **MUST** be errors.

Decision 3.4 (Compile the normal form, not the tree). The compiler's input is **k1ndling_norm::Norm**, with the sole exception of Requirement 3.3. Compiling the tree would make the output depend on how the author happened to write a parallel composition, and Obligation 6.2 would be false.

4 The target calculus

4.1 Formation, equations, rules

From [3, §3.2], with $a, b, c, f : \mathbf{N}$ and $p, q : \mathbf{P}$:

$$\begin{array}{lll}
\mathbf{m}(a, v) & \mathbf{b}_l(a, b) & \mathbf{cons}_A(a_1, \dots, a_n, f) \\
\mathbf{d}(a, b, c) & \mathbf{b}_r(a, b) & \mathbf{cons}_l(a, b, c) \\
\mathbf{k}(a) & \mathbf{s}(a, b, c) & \\
\mathbf{fw}(a, b) & \mathbf{e}(a) & \\
\mathbf{q}(a, p) & \mathbf{0}, p \mid q & @p : \mathbf{N}, *a : \mathbf{P}
\end{array}$$

Requirement 4.1 (The drop and the guard are distinct constructors). $*-$ and **e** **MUST** be distinct constructors of the target type and no code path **MAY** convert one into the other. [3, Prop. 3.5] needs the conflation as a hypothesis; with them separate, an arbitrary running process cannot match a guard however many equations the theory carries. A representation sharing one tag between them **MUST NOT** be used even with a discriminating flag.

Definition 4.2 (Equations, [3] Def. 3.1). $(P, |, \mathbf{0})$ is an abelian monoid; $@*a = a$; and, under A , $*@p = p$.

Requirement 4.3 (Drop elimination). Orient the equations left to right. By [4, Lem. 2.8] the system is terminating and confluent and a closed normal form contains no $*-$. The smart constructors **MUST** apply both rewrites at construction, so that every **Term** in existence is in normal form and every traversal may assume a flat multiset of atoms. Under **presentation-b**, Requirement 4.14 inverts this.

Definition 4.4 (Rules, [3] Def. 3.3 and 3.8).

$$\begin{aligned}
\text{dist} &: d(a, b, c) \mid m(a, v) \longrightarrow m(b, v) \mid m(c, v) \\
\text{kill} &: k(a) \mid m(a, v) \longrightarrow \mathbf{0} \\
\text{forward} &: fw(a, b) \mid m(a, v) \longrightarrow m(b, v) \\
\text{bind}_l &: b_l(a, b) \mid m(a, v) \longrightarrow fw(v, b) \\
\text{bind}_r &: b_r(a, b) \mid m(a, v) \longrightarrow fw(b, v) \\
\text{synch} &: s(a, b, c) \mid m(a, v) \longrightarrow fw(b, c) \\
\text{eval} &: e(a) \mid m(a, v) \longrightarrow *v \\
\text{quote} &: fw(a, b) \mid q(a, p) \longrightarrow m(b, @p) \\
\text{make}_l &: \text{cons}_l(a, b, c) \mid m(a, @p) \mid m(b, @r) \longrightarrow m(c, @(p \mid r)) \\
\text{make}_A &: \text{cons}_A(a_1, \dots, a_n, f) \mid m(a_1, v_1) \mid \dots \mid m(a_n, v_n) \longrightarrow m(f, @A(\vec{v}))
\end{aligned}$$

closed under parallel composition and the equations, subjects matched up to $\equiv_{\mathcal{N}}$.

Requirement 4.5 (The rule table is generated, not written). With make_A a schema over atom shapes, the rule table **MUST** be *derived* from the shape table: one row per shape, giving arity and argument sorts, from which both the atom and its constructor are generated. Hand-writing thirteen-plus rows invites exactly the divergence [3, Rem. 3.4] records. The machine's step function **MUST** be a dispatch over the generated table.

Requirement 4.6 (The three departures). [3, Rem. 3.4]'s corrections **MUST** each carry a test. The binders produce forwarders: $b_l(a, b) \mid m(a, v) \longrightarrow fw(v, b)$ and $b_r(a, b) \mid m(a, v) \longrightarrow fw(b, v)$. A version producing a message duplicates **forward**, collapses a lattice edge, and breaks occurrence kinds (o1) and (o2). **fw** is binary. A constructor's delivery channel is in its own argument list and distinct from its inputs.

4.2 The constructor family

[3, Def. 3.8] replaces the four constructors of the previous revision with one per atom shape, and [3, Rem. 3.13] makes the enlargement compulsory rather than fastidious: phase two erects allocated atoms by Remark 3.11, which calls for a constructor at every shape the compiled output contains.

Requirement 4.7 (The family is computed, and is one level deep). The family **MUST** be computed, per program, as

$$\mathcal{F}(P) = \{ \text{cons}_A : A \in \text{shapes}(\llbracket P \rrbracket), \text{ some atom of shape } A \text{ in } \llbracket P \rrbracket \text{ mentions a bound name} \}$$

where $\llbracket P \rrbracket$ is the phase-one image. It **MUST NOT** be taken as the transitive closure of the signature under **cons**. [3, Rem. 6.15] now states why the recursion terminates at one level per program: **ERECT** erects only phase-one atoms, so the shapes needed are those of atoms mentioning a bound name, and the cons_A atoms doing the erecting appear in the phase-two image, which is never itself erected. The compiler **MUST** emit $\mathcal{F}(P)$ in the artefact header, since Requirement 8.7 and the lattice report both consume it.

Requirement 4.8 (cons_q and the shapes with a process argument). [3, Def. 3.8] now states the member for a shape with a process argument, $\text{cons}_q(a, b, c) \mid m(a, p_1) \mid m(b, p_2) \longrightarrow m(c, q(@p_1, p_2))$, in presentation B's sorting, with [3, Rem. 3.9] giving A's reading $@q(v_1, *v_2)$. The implementation **MUST** provide it in the sorting of the selected presentation, generated from the shape table like every other member (Requirement 4.5), and under A **MUST** apply drop elimination to the result so that the erected atom is in normal form (Requirement 4.3). The two readings **MUST** produce the same erected atom on the corpus, which is a cheap check on the coercion and is the executable form of [3, Rem. 3.9].

Requirement 4.9 (Erecting a storage atom whose payload is allocated). [3, Ex. 6.9] erects $q(b, e(c))$, in which the payload itself mentions an allocated name. The compiler **MUST** therefore build the payload first — cons_e at c 's chain, yielding $@e(c)$ — and feed that name to cons_q , which means cons_e is in $\mathcal{F}(P)$ for every unit containing an input, independently of Requirement 8.7. The family computation **MUST** include the shapes reached through gate payloads and not only the shapes of top-level atoms.

4.3 Representation, encoding, hashing

Requirement 4.10 (Encoding). A target term **MUST** have a canonical byte encoding built bottom-up in the smart constructors and cached on the node, in the style of [5, §6]: tag byte, then LEB128 lengths and counts, then children. Parallel components **MUST** be ordered by bitwise comparison of their own encodings, with **0** absent. The content hash **MUST** be BLAKE2b-256 of the encoding.

Requirement 4.11 (Tags: a shape table, not a flat enumeration). Target tags **MUST** occupy a block disjoint from the source tags of [5, §6]. Because the constructor family is a schema, the tag space **MUST** be structured as a shape byte plus a one-bit constructor flag, so that cons_A 's tag is derived from A 's and a new shape costs no tag negotiation. The reserved layout:

Tag	Shape
0x30	0
0x31	par, $n \geq 2$ components
0x32–0x3B	the atoms: m, d, k, fw, b_l, b_r, s, e, q , reserved
0x40	$@p$, name former
0x41	$*a$, presentation-b only; rejected by the decoder under A
0x50–0x5B	cons_A , where A is the shape at $0x30 + (t - 0x50)$; cons_l is 0x51

Requirement 4.12 (Name equivalence is a hash comparison). Under A, $a \equiv_{\mathcal{N}} b$ iff their encodings are equal iff their content hashes are equal modulo collision ([4, Prop. 2.10]). The machine **MUST** match subjects by content hash and **MUST NOT** implement a separate $\equiv_{\mathcal{N}}$ procedure. This discharges corrigendum 5 of [3, §11].

Requirement 4.13 (No variables in the target). The *target* node type **MUST NOT** have a bound-variable, free-variable or wildcard case, nor a **new** case. The binder lives in the intermediate representation of Section 5.2 and nowhere else; a phase-two image that still carries one is a compiler bug, and the type system **MUST** make it unrepresentable.

Requirement 4.14 (Under B, the opposite on drops). With **presentation-b**, $*@p \rightarrow p$ **MUST NOT** be applied, the drop node **MUST** be retained, every traversal **MUST** carry a descent case for it ([4, Prop. 2.4(iii)]), and the term type **MUST NOT** be routed through `kindling_norm::Norm::eval`.

5 The two-phase translation

5.1 The bug, and why the specification had it too

[3, §6.1] exhibits

$$W = \text{for}(z \leftarrow b)(*z \mid *z) \mid b!(R) \mid a!(X) \mid a!(Y), \quad R = \text{for}(y \leftarrow a)y!(*y),$$

on which a translation whose apparatus names are functions of syntactic position is unsound: **e** instantiates the same compiled text at both drop sites, so both instances of R carry the same proxy c , and after the links fire nothing pairs a forwarder with the message from its own copy. A run yields $\llbracket @X!(Y) \rrbracket \mid \llbracket @Y!(X) \rrbracket$, which an encoded observer distinguishes from the source.

Remark 5.1 (v0.2 was a positional scheme). This specification is among the affected artefacts, and it is worth being exact about how. v0.2’s Decision 4.6 made the seed a function of the source name, $\llbracket @P \rrbracket := @\llbracket P \rrbracket_{@P}$, and Requirement 4.7 allocated seed powers by canonical path within a unit. Both were aimed at *canonicity* — the same source must compile to the same bytes — and they achieved it. Canonicity is precisely what makes the bug bite: if the same source always yields the same apparatus names, then two run-time instances of one compiled occurrence necessarily share them. The invariant v0.2 secured and the invariant correctness needs were in direct opposition, and v0.2 did not notice. The two-phase translation resolves the opposition by moving apparatus names out of the compiled artefact entirely: the artefact is closed, and freshness happens at run time (Section 6).

Requirement 5.2 (W is the first regression test). W MUST be a golden test in **f1r3comb-check** before any other translation test, run under both B3LL0WS and the machine, asserting the observation at $\llbracket @X \rrbracket$. It is the term the translation exists to get right, and any future change to naming MUST be measured against it.

5.2 Phase one

Definition 5.3 ([3] Def. 6.3). On names, $\llbracket @P \rrbracket := @\llbracket P \rrbracket$. On processes,

$$\begin{aligned} \llbracket 0 \rrbracket &= 0 \\ \llbracket P \mid Q \rrbracket &= \llbracket P \rrbracket \mid \llbracket Q \rrbracket \\ \llbracket x!(P) \rrbracket &= \mathbf{m}(\llbracket x \rrbracket, @\llbracket P \rrbracket) \\ \llbracket \text{for}(y \leftarrow x)P \rrbracket &= (\text{new } p_0, \dots, p_k, c_1, \dots, c_k) (D(\llbracket x \rrbracket; \vec{p}) \mid G(p_0, B) \mid L_1 \mid \dots \mid L_k) \end{aligned}$$

with $G(p, S) := (\text{new } b, c)(s(p, b, c) \mid q(b, S) \mid e(c))$, D the duplicator chain under $(\text{new } q_1, \dots, q_{k-1})$, $o_1, \dots, o_k$ the free occurrences of y in P , and L_i and the residue in B given by Table 1.

Requirement 5.4 (Phase one is a homomorphism, and that is the point). Phase one MUST be implemented as a structural homomorphism on the source normal form, with no name allocation of any kind: bound names are binder occurrences in the intermediate representation, not names. It is therefore canonical without argument (Obligation 6.2), and the entire freshness question is deferred to phase two, where it belongs.

Requirement 5.5 (The intermediate representation). RC'' MUST have its own Rust type in its own crate, with a **new** node and de Bruijn indices for its bound names — not a **Term** with an extra variant, because Requirement 4.13 must remain enforceable by the type system. The IR MUST support scope extrusion and α -equivalence, and MAY be encoded and hashed for testing, but MUST NOT be deployable.

Requirement 5.6 (Occurrence enumeration order). [3, Def. 6.3] says only “let $o_1, \dots, o_k$ enumerate the free occurrences”. The order decides which bound name is c_1 , hence which leaf it is allocated in phase two, hence the bytes of the output. Occurrences **MUST** be enumerated by ascending byte offset in the encoding of the receive’s body, which is a function of the source normal form and therefore canonical.

Requirement 5.7 (A unit is a quotation). Scope extrusion cannot move a **new** out of a quotation, and the body B of an input travels as the payload of $\mathbf{q}(b, B)$. Phase two is therefore applied *per quotation*: the top-level term is one unit, and the body of each input is another, compiled on its own. [3, Def. 6.8]’s “bound names $z_1, \dots, z_n$ under a single **new**” is to be read as the binders of one unit after extrusion within that unit.

5.3 Phase two

Definition 5.8 (Allocation, [3] Def. 6.6). Fix $K := @m(@0, @0)$, $L\sigma := @m(\sigma, @0)$ and $R\sigma := @m(\sigma, K)$. Then

$$\text{ALLOC}(t; t_L, t_R) := \mathbf{d}(t, u_1, u_2) \mid \text{cons}_m(u_1, z_0, t_L) \mid m(z_0, @0) \mid \text{cons}_m(u_2, z_1, t_R) \mid m(z_1, K).$$

L, R are injective with disjoint images ([3, Lem. 6.7]), so $w \mapsto \sigma \cdot w$ is injective.

Requirement 5.9 (The gadget’s channels are per-node). [3, Rem. 6.13] now states it: within a single instance the allocation gadget’s internal channels **MUST** be distinct per node of the tree. Written with one shared set, a σ put at u_1 by the root gadget’s $\mathbf{d}$ may be consumed by a child gadget’s cons_m , which delivers $L\sigma$ where $LL\sigma$ was intended, and the tree deadlocks or mis-addresses — within one instance, with no concurrency involved. The naming **MUST** follow Requirement 6.4, which makes the per-node distinctness a consequence of the naming function rather than a separate rule to remember.

Definition 5.10 ([3] Def. 6.8). For a unit with bound names $z_1, \dots, z_n$,

$$[[P]]^\# := \text{ALLOC-tree}(r; t_1, \dots, t_n, t_{\text{next}}) \mid \text{ERECT}(t_1, \dots, t_n),$$

where r is the static channel at which the instance receives its address, and **ERECT** contains, for each atom $A(z_{i_1}, \dots, z_{i_m})$ of $[[P]]$, the pair $\text{cons}_A(t_{i_1}, \dots, t_{i_m}, f_A) \mid \mathbf{e}(f_A)$ of [3, Rem. 3.11]. Atoms with no bound name are emitted unchanged.

Requirement 5.11 (One spare leaf per nested unit). [3, Rem. 6.13] now states it: one spare suffices for a self-instantiating unit such as [3, Ex. 6.9], and a unit with nested inputs needs one per nested unit, each with its own static r_j at which the released instance receives its address. The allocation tree **MUST** therefore produce one spare leaf per directly nested unit, plus one for self-instantiation where the unit can re-erect itself, and **ERECT** **MUST** emit $m(r_j, t_{\text{spare}, j})$ for each. A unit compiled with too few spares stalls the first time a nested input is released, which no single-input example exhibits.

Requirement 5.12 (Fan-out of the address). Each leaf is consumed once per atom that mentions its bound name, so a bound name occurring in m erected atoms needs m copies of its leaf message, supplied by a duplicator chain at t_i . The count **MUST** be computed from **ERECT** and **MUST NOT** be guessed; an under-supplied leaf stalls erection, and an over-supplied one leaves a message that a later instance may consume, which is a correctness failure and not a leak.

5.4 The permutation discipline

[3, Rem. 6.10], on which v0.3 declined to build, is now [3, Def. 6.10] — the single-address-input discipline — together with [3, Lem. 6.11], which proves permutation invariance for images satisfying it. The claim is no longer a remark, and the implementation’s job changes accordingly: from testing a hope to conforming to a condition.

Definition 5.13 ([3] Def. 6.10). A phase-two image satisfies the *single-address-input discipline* if every constructor placed at a statically named channel has at most one input carrying an address, and every join of two or more addresses occurs at a channel derived from a single address.

Requirement 5.14 (Conformance is mandatory and mechanically checked). Every phase-two image the compiler emits MUST satisfy Definition 5.13, and **f1r3comb-lower** MUST verify it as a post-pass rather than rely on the emitter being correct. The check is a dataflow over the emitted term: mark every channel as address-derived or static, propagate through constructor outputs, and assert that no statically-marked constructor has two address-marked inputs. It MUST run in the default build and MUST fail compilation, not warn: by [3, Rem. 6.12] an implementation that ignores the question is wrong, not merely unproved.

Requirement 5.15 (ERECT as stated does not satisfy the discipline). This is the practical consequence and it is worth stating as a requirement rather than a remark, because it decides the construction. ERECT of [3, Def. 6.8] emits $\text{cons}_A(t_{i_1}, \dots, t_{i_m}, f_A)$ with the t_i static channels carrying leaves, so for every atom mentioning two or more bound names it has $m \geq 2$ address-valued inputs at static channels and violates Definition 5.13. That is most atoms: the distributor's $\text{d}(x, p_0, q_1)$ and the gate's $\text{s}(p_0, b, c)$ both do. The allocation tree is unaffected — each cons_m there takes one address and one constant. So the implementation MUST NOT take [3, Def. 6.8] literally, and MUST take the scheme of [3, Rem. 6.12] or another conforming one.

Requirement 5.16 (Two-stage erection). [3, Rem. 6.12] says to thread a single copy of the address through single-input chains and perform every join at an address-derived channel. A join at an address-derived channel needs an atom standing at that channel, and an atom stands at a computed name only by being erected there ([3, Rem. 3.11]), so the construction is necessarily two-stage and the implementation MUST emit it as such:

- (i) **Bootstrap.** From the static r , a chain of single-input cons_m applications builds the leaves $\sigma \cdot w$ for the paths the unit needs, each step taking one address and one constant. Every constructor here has exactly one address input, so this stage conforms by construction.
- (ii) **Erector.** The bootstrap erects, at address-derived channels, the cons_A atoms that build the unit's own atoms. These may join freely, because their channels are leaves of one address and by [3, Lem. 6.7] no other instance's chain can deliver to them.
- (iii) **The unit.** The erector erects the unit's atoms at its leaves.

Paths MUST be of one fixed length, so that the path functions are prefix-free and injective and leaf distinctness within a unit follows from injectivity alone rather than from the order in which the chains ran.

Requirement 5.17 (The construction is recorded). The artefact header MUST name the erection construction, because two conforming schemes differ in their reduction sequences and hence in every golden trace and every step count. A golden file MUST NOT be compared across constructions.

Remark 5.18 (What the discipline costs). [3, Prop. 8.1] now folds the factor in: conforming by [3, Rem. 6.12] multiplies the phase-two image by a factor logarithmic in the number of allocated names, the depth of the chains. v0.3 estimated the same figure for the same construction, and Requirement 8.9 carries it.

5.5 Freshness

Requirement 5.19 (Root address, by size). [3, Lem. 6.14] replaces the shape argument `v0.3` proposed — which the total constructor family had closed off — with a size argument: if σ is $\equiv_{\mathcal{N}}$ -normal and larger than every $\equiv_{\mathcal{N}}$ -normal name occurring in $\llbracket P \rrbracket$, then no leaf beneath σ occurs in $\llbracket P \rrbracket$, since **L** and **R** strictly increase size. The deploy path **MUST** therefore choose a root address whose normal form exceeds the largest name of $\llbracket P \rrbracket$, **MUST** compute that bound from the phase-one image rather than assume it, and **MUST** record it in the artefact header so a later deploy can re-check it without recompiling. This is materially cheaper to implement than a shape test and cheaper to check than a closure computation, and it is the part of the revision that most simplifies the machine.

Requirement 5.20 (The second clause is a compiler obligation). [3, Lem. 6.14]’s second clause — that no name built at run time by an `(o5)` chain is a leaf — holds because such chains build from names received on source channels, and addresses never enter the payload of a source message. The paper notes this is an obligation on the translation rather than a property of the calculus, and mechanically checkable. The compiler **MUST** check it, as a second marking pass: no address-derived channel may supply a *value* to a constructor of an `(o5)` chain. The check **MUST** be scoped to post-erection flows, because during erection leaves are precisely what travel as values — that is what **ERECT** does — so a naive reading of the clause is contradicted by the construction that establishes it.

Erratum 5.21 (No freshness condition on the static channels). [3, Lem. 6.14] gives the leaves freshness against $\mathcal{N}^*(\llbracket P \rrbracket)$ and [3, Lem. 7.6] needs an encoded context never to name the apparatus. Neither covers the *static* channels of [3, Def. 6.8]: r , the gadget channels, the f_A . Nothing states that r differs from an encoding of a source name. If it does not, an encoded context can post a name at r , and an instance will allocate beneath an address of the context’s choosing — which defeats [3, Lem. 6.11] at its hypothesis, since the two addresses need no longer be prefix-incomparable, and defeats [3, Lem. 6.14] at its, since the address need no longer be large. The fix is the same size argument one line over. Proposed amendment: state in §6.3 that the static channels, like the root address, are chosen larger than every name of $\llbracket P \rrbracket$, and extend [3, Lem. 7.6] to cover them.

Requirement 5.22 (Static channels are fresh too). The static channels **MUST** satisfy the same size condition as the root address, and `f1r3comb-lower` **MUST** assert it. A deploy **MUST** reject an artefact whose static channels do not, with `comb-static-not-fresh`, because an encoded context that can name r controls the apparatus.

Requirement 5.23 (Distinct deploys get incomparable addresses). Two deploys of the same program share every static channel, so they are two instances and **MUST** be given prefix-incomparable root addresses — the hypothesis of [3, Lem. 6.11]. The machine **MUST** maintain the address allocator and refuse a root address comparable with one in use. Prefix comparison is a structural test on the **L/R** spine, linear in the name.

6 Canonicity

6.1 What survives, and what it now costs

[1, § identity]: `CAMPF1R3` keys a channel by the content hash of its encoding, so a compiled form may not be a channel key unless the compiler is canonical. In `F1R3COMB` compiled code is always a key, since $\llbracket @P \rrbracket = @\llbracket P \rrbracket$.

Requirement 6.1 (Canonicity). If $P \equiv Q$ then the encodings of $\llbracket P \rrbracket^\sharp$ and $\llbracket Q \rrbracket^\sharp$ are byte-equal. Compilation **MUST** be a function of the source normal form alone.

Obligation 6.2 (Canonicity theorem). Phase one is a structural homomorphism with no name allocation (Requirement 5.4), so it is canonical given a canonical source normal form ([5, Prop. 6.9]), modulo the enumeration order of Requirement 5.6 and the α -equivalence of the IR. Phase two introduces only static channels, fixed by Requirement 6.4. Hence the result. This **MUST** be proved and tested (Obligation 11.3).

Remark 6.3 (The tension of Remark 5.1 is gone). Under the two-phase translation, canonicity and per-instance freshness no longer pull against each other, because they act on different objects: canonicity is a property of the compiled artefact, which contains no apparatus name at all, and freshness is a property of the address an instance receives, which is not in the artefact. This is the structural reason the two-phase translation is the right shape, and it is worth stating because v0.2’s difficulty was not a slip but a consequence of trying to have both in one object.

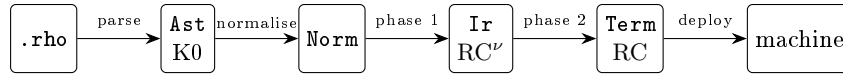
Requirement 6.4 (Static channel naming). The static channels of the phase-two image — r and the nested units’ r_j , the per-node gadget channels of Requirement 5.9, the f_A , and the intermediate channels of and the bootstrap channels of Requirement 5.16 — **MUST** be named by a canonical function of their position in the phase-one image and their role, and **MUST NOT** be allocated from a counter whose value depends on traversal order elsewhere. They are deliberately shared between instances, so they need no freshness guarantee against each other; they **MUST** nonetheless satisfy the size condition of Requirement 5.22, and the naming function **MUST** make per-node distinctness within an allocation tree (Requirement 5.9) a consequence of the function rather than a separate rule.

Remark 6.5 (Two programs, one machine). Static channels are also shared between *different* programs deployed to one machine, if their naming functions collide. Requirement 6.4 therefore **SHOULD** derive statics from the content hash of the unit’s phase-one image together with the role, which makes collision between different units improbable and between instances of the same unit certain — which is what [3, Rem. 6.10] intends.

7 Architecture

Crate	What it is	Depends on
<code>f1r3comb-term</code>	The target: sorts, shape table, generated atom and constructor types, drop elimination, encoding, hashing, printer, decoder, atom multiset	<code>kindling-norm</code> (hash only)
<code>f1r3comb-ir</code>	RC'' : the binder, de Bruijn, scope extrusion, α -equivalence	<code>f1r3comb-term</code>
<code>f1r3comb-lower</code>	Phase one (source $\rightarrow$ IR) and phase two (IR $\rightarrow$ target): checks, occurrence analysis, chains, allocation, erection, the family computation	<code>kindling-ast</code> , <code>kindling-norm</code> , <code>f1r3comb-ir</code>
<code>f1r3comb-space</code>	CAMPF1R3 instantiated over target terms	<code>campf1r3-core</code> , <code>f1r3comb-term</code>
<code>f1r3comb-run</code>	The machine: address allocator, deploy, generated rule table, scheduler, event stream	<code>f1r3comb-space</code>
<code>f1r3comb-check</code>	Differential testing against B3LL0WS, the interleaving tests, lattice audit, golden files	all of the above, <code>b3ll0ws</code>

Requirement 7.1 (Independent usability). `f1r3comb-term` MUST be usable without the compiler, because the occupancy checker of [4, §6] wants a term type, an encoding and an atom multiset and nothing else. `f1r3comb-ir` MUST be usable without `f1r3comb-run`, so that phase one can be inspected on its own — which is where the lattice report is informative (Requirement 10.1). No crate MAY depend on `f1r3comb-check`.



F1R3X gains `comb`, `comb-ir`, `comb-hash`, `comb-run` and `comb-lattice`. `comb-run` MUST reproduce the core-count invariance `run` already has.

8 The lowering

8.1 Phases

1. Level check on the tree; guard check on the tree (Requirement 3.3); normalise; closedness check.
2. Phase one, per Definition 5.3, producing IR.
3. Compute $\mathcal{F}(P)$ (Requirement 4.7) and the leaf demand (Requirement 5.12).
4. Phase two, per unit (Requirement 5.7), producing a closed target term.

	occurrence of y	in the quoted body	link
(o1)	input subject $\text{for}(z \leftarrow y)S$	subject $\rightarrow c_i$	$\mathbf{b}_l(p_i, c_i)$
(o2)	output subject $y!(S)$	subject $\rightarrow c_i$	$\mathbf{b}_r(p_i, c_i)$
(o3)	forwarded name $v!(*y)$	atom deleted	$\mathbf{fw}(p_i, \llbracket v \rrbracket)$
(o4)	drop $*y$ in process position	$\rightarrow \mathbf{e}(c_i)$	$\mathbf{fw}(p_i, c_i)$
(o5)	inside a quotation $@S, y$ free in S	subject $\rightarrow c_i$	constructor chain for S

Table 1: The five occurrence kinds, [3, Table 1].

5. Encode, hash, and emit the artefact header: presentation, feature set, erection construction, $\mathcal{F}(P)$, and the shapes available for a root address.

Requirement 8.1 (Total and terminating, no Rust recursion). Both phases **MUST** be total on terms passing the checks, **MUST** terminate, and **MUST NOT** recurse on the Rust stack, following the workspace rule.

8.2 Occurrence analysis

Requirement 8.2 (Classification). For a receive with body B , the analysis **MUST** collect every $\mathbf{Name}::\mathbf{Bound}(d)$ whose index equals the number of binders between it and the receive, and assign each exactly one kind by this disjoint test on its immediate role: the **chan** of a **Receive** bind, not under an intervening quotation, is **o1**; the **chan** of a **Send**, likewise, is **o2**; the name of an **Eval** that is the sole argument of a **Send** whose channel is not the occurrence is **o3**; the name of an **Eval** in process position is **o4**; anything under an intervening $\mathbf{Name}::\mathbf{Quote}$ is **o5**, with the chain built for the outermost such quotation. The tests **MUST** be applied in that order. An occurrence matching none **MUST** raise an internal error naming the node.

Requirement 8.3 (Coincident kinds). [3, Table 1]’s note about several kinds coinciding in one atom is about occurrences, not kinds: each occurrence has one kind. The implementation **MUST** treat coincident occurrences independently, each with its own bound name and link, in the order of Requirement 5.6, and **MUST NOT** fuse two occurrences into one proxy. $R = \text{for}(y \leftarrow a)y!(*y)$ of §5.1 is exactly this case, and is the term on which the old scheme failed; fusing would reintroduce a shared name within a single instance, which no amount of per-instance allocation repairs.

8.3 Constructor chains for (o5)

Requirement 8.4 (Chain construction). For an (o5) occurrence inside the outermost enclosing quotation $@S$, the compiler **MUST** emit the chain rebuilding $@[S]$ from the received name and delivering it at c_i , by recursion on the path from the occurrence to the root of S , per [3, Prop. 3.12]: the occurrence itself needs no constructor; a parallel composition needs **cons_p**; an output needs **cons_m**; an input needs **cons_d** when its bound name occurs in its body and **cons_s** when it does not; every node off the path is static and supplied as a constant message. Chains **MUST** be built outermost-last.

Remark 8.5 (Proposition 3.12 is conditional, and its condition holds here). [3, Prop. 3.12] is stated for static apparatus, and [3, Rem. 3.13] withdraws it for phase two. The distinction matters for the implementation: the four constructors suffice for the (o5) *chains*, which rebuild source structure and whose shapes are those of the source, while the *erection* of phase two needs $\mathcal{F}(P)$. These are two different demands on the family and Requirement 4.7 takes their union.

Requirement 8.6 (No repeated subject in a constructor). A constructor **MUST NOT** be emitted with two equal argument channels, nor with a delivery channel among its inputs ([3, Rem. 3.4]). Where the same value is needed twice, the compiler **MUST** split with **d** first. A join group with a repeated channel is not a request for two data at that channel.

8.4 Shortcuts

Requirement 8.7 (Feature-gated, and they enlarge the family). [3, Rem. 6.4] gives three cheaper cases and states, following [3, Rem. 3.13], that they are not free: they put a bound name into the argument of an `e` or a `fw`, so they add `conse` and `consfw` to $\mathcal{F}(P)$. Under [3, Lem. 6.14] this no longer threatens freshness, which is by size rather than by shape, but it does change $\mathcal{F}(P)$, the lattice point and every content hash. The feature `MUST` be off by default and recorded in the artefact header.

8.5 Worked examples

Requirement 8.8 (Three golden traces). The following `MUST` be golden files — source, IR, target, encoding, hash, and the full reduction sequence with step counts:

- (i) W of §5.1, the term the two phases exist for, asserting the observation at $\llbracket @X \rrbracket$ and, explicitly, the *absence* of the crossed residue (Requirement 5.2);
- (ii) $\llbracket W \rrbracket$ at phase one, asserting that the two instances of R are α -distinct ([3, Ex. 6.5]);
- (iii) $D_x = \text{for}(y \leftarrow x)(x!(*y) \mid *y)$, the recursion combinator, whose run [3, Ex. 6.9] works out in full: seven bound names, the erection, the eight reduction steps, and the residue at a strictly deeper address.

Goldens `MUST` be regenerated only by an explicit command.

Requirement 8.9 (Output size). $\llbracket P \rrbracket^\# = O(|P| \log |P|)$ under the conforming construction of Requirement 5.16, the logarithm being the chain depth, which [3, Prop. 8.1] now folds into its statement. On the nesting family of [3, Prop. 8.2] phase one `MUST` give $3d + 1$ atoms against Yoshida’s $(3^{d-1} + 1)/2$, tested as a law parameterised over d ; phase two multiplies by a constant, which `MUST` be measured and reported rather than asserted.

9 The machine

9.1 The instantiation

Every rule of Definition 4.4 consumes messages at one to n named subjects and produces messages or atoms. There is no pattern to match beyond the subject, no binder to open, no substitution. `CAMPF1R3` is exactly that store, with joins for the multi-subject case.

Decision 9.1 (The machine is `CAMPF1R3` with a three-line matcher). `f1r3comb-run` `MUST` be `campf1r3-core::Space` instantiated over target terms and `MUST NOT` implement its own indexing, joins or replay. The spatial matcher of `kindling-campf1r3` is not used and `MUST` not be linked.

CAMPF1R3 parameter	F1R3COMB instance
C , channel	<code>CName</code> , a target name, hashed by its encoding
A , datum	<code>enum Val { Loud(CName), Quiet(Term) }</code> — the sorts of <code>m</code> and <code>q</code> differ under A , so an enum over two types, not a struct with a flag
P , pattern	<code>Acc { accepts_quiet: bool }</code>
K , continuation	<code>Rule</code> , from the generated shape table
M , matcher	reject a quiet datum for a non-forwarder pattern; otherwise accept
O , observer	<code>NoObserver</code> at <code>v1</code>

Requirement 9.2 (Joins are variable-arity). With cons_A a schema, a constructor’s join group has $\text{ar}(A)$ channels. The deploy path **MUST** issue `consume(channels, patterns, k)` with the group read off the generated table, and **MUST NOT** assume two or three. CAMPF1R3 supports joins of arbitrary group size; the implementation **MUST** nonetheless bound the arity and diagnose a shape whose arity exceeds it.

Requirement 9.3 (`eval` normalises before it deploys). Under A the rule produces $*v$, whose normal form is the process v quotes. The machine **MUST** construct $*v$ through the smart constructor, which performs the elimination, and deploy the result. It **MUST NOT** short-circuit by projecting the payload out of the name: the short-circuit is observationally identical here and is the step that becomes unsound under **presentation-b**.

Requirement 9.4 (Linear, never persistent). Every `consume` **MUST** pass `persist = false` and an empty `PeekSet`; `install` **MUST NOT** be used. Every rule consumes its participants, and there is no persistent atom in the calculus.

9.2 Addresses

Requirement 9.5 (The address allocator). The machine **MUST** own an address allocator satisfying Requirements 5.19 and 5.23: it hands each deploy a root address of a shape outside that program’s family, prefix-incomparable with every address in use, and posts it at the unit’s r . It **MUST** record addresses in the event stream, because an address is the only thing that distinguishes two instances and a trace without them is unreadable.

Hazard 9.6 (Addresses grow without bound). Every self-instantiation descends one level: [3, Ex. 6.9]’s residue sits at a strictly deeper address, and $|\text{L}\sigma| = |\sigma| + O(1)$. A recursive program therefore grows its channel keys linearly in the number of unfoldings, and a program that builds names from constructed names doubles them. [3, §12] open question 3 and [4, §7] both name this; under phase two it is no longer a pathological case but the normal behaviour of every loop.

Requirement 9.7 (Interning and budget). Constructed names **MUST** be hash-consed, with the encoding stored once and channels referring to it by index, and the representation **SHOULD** share the common prefix of an address spine so that descending one level costs $O(1)$ rather than $O(|\sigma|)$. The machine **MUST** accept a budget — maximum name size, maximum distinct names — and halt with `comb-name-budget` naming the constructor or the unfolding that exceeded it. Because Hazard 9.6 now fires on ordinary recursive programs, the budget **MUST** be a supported operational parameter and not a debugging aid, and the measured growth rate **MUST** be reported by `comb-run -trace`.

Requirement 9.8 (Races are scheduler choices and **MUST** be visible). [3, §3.3] records two deliberate races, and phase two adds a third: every cross-match at a static channel between concurrent instances ([3, Rem. 6.10]). The machine **MUST** emit a trace event at each resolution naming the alternatives not taken and the addresses involved. This is not only for RHOBOTS: it is the instrumentation Obligation 11.2 needs, since a counterexample to permutation invariance will appear first as a cross-match followed by a divergence.

10 The lattice audit

Requirement 10.1 (Report both phases). `f1r3comb-lower` **MUST** return the atom multiset of *both* images, and `f1r3x comb-lattice` **MUST** print both points. The phase-two point is nearly uninformative — every compiled unit erects, so every compiled unit carries most of $\mathcal{F}(P)$ and sits high on the constructor axis of [3, §9] — while the phase-one point is the one that says what the source term needs, and is the one the lattice was drawn for. Reporting only the target would make the audit look like a constant function and would waste [4, Thm. 6.5].

Requirement 10.2 (The multiset is the logic’s input). `f1r3comb-term` MUST expose the atom multiset and the encoding through a stable API so the occupancy checker of [4, §6] can be built against it, preserving the `m/q` sort distinction on which [4, Rem. 2.5] turns. It SHOULD NOT expose more.

Requirement 10.3 (The weighted count). Where the implementation computes the halting bound of [3, Prop. 9.9] — in the budget of Requirement 9.7, in the lattice report, or in a test — it MUST use the *weighted* count of [4, Def. 6.6], in which `bl`, `br` and `s` count twice, because those rules consume themselves and produce an `fw` and so preserve rather than decrease the plain count. The unweighted count bounds the weighted one only by a factor of two.

11 Correctness obligations and testing

Obligation 11.1 (The *W* regression). Requirement 5.2. *W* run under the machine MUST produce the observation the source produces, and MUST not produce the crossed residue. A variant with three messages at *a* and a body of three drops MUST also be tested, since two instances can be crossed by accident and three cannot.

Obligation 11.2 (Discipline conformance, and permutation tested anyway). [3, Lem. 6.11] proves permutation invariance for images satisfying [3, Def. 6.10], so the primary evidence is now the conformance check of Requirement 5.14, which MUST be a unit test on every corpus term and not only a compile-time assertion. The behavioural test remains worth running, because conformance is a property of the emitted term and the machine could still break the lemma’s hypothesis — by handing out comparable addresses, or by consuming a leaf twice. So: for a corpus of terms with $k \geq 2$ concurrent instances of one unit, run under a scheduler that *maximises* cross-matching at static channels — an adversarial profile that, at each choice point, prefers the datum from the instance that did not produce it — and compare observations against B3LL0WS. The deterministic profile will rarely exhibit the interleaving; a test that only runs deterministically tests nothing here. The adversarial profile MUST be a supported CAMPF1R3 strategy in `f1r3comb-check`. A term that fails this while passing Requirement 5.14 is a counterexample to [3, Lem. 6.11] and MUST be reported as such rather than patched.

Obligation 11.3 (Canonicity). For randomly generated K0 terms and $\equiv$ -variants obtained by permuting parallel components and re-associating, the compiled encodings MUST be byte-equal, at both phases.

Obligation 11.4 (The conflation regression). A test MUST assert that $Q \mid m(@Q, @P)$ is stuck for a nontrivial *Q* — that the machine does not implement [3, Prop. 3.5] — and a second MUST assert rigidity: no process is equal to an `e` atom unless written as one.

Obligation 11.5 (Freshness). Three checks, all cheap under [3, Lem. 6.14]. The root address exceeds the largest name of $\llbracket P \rrbracket$ (Requirement 5.19); the static channels do too (Requirement 5.22); and no address-derived channel supplies a value to an `(o5)` chain (Requirement 5.20). A property test MUST include the adversarial source term $@(x!(\mathbf{0}))$, whose encoding is $L[\llbracket @x \rrbracket]$ — a leaf of the address $\llbracket @x \rrbracket$ — since it passes any test that checks only distinctness of leaves from one another, and is excluded here only by the size condition.

Obligation 11.6 (Operational correspondence). [3, Lem. 7.7] fixes the reduction sequence after a communication. The machine MUST be tested against the step counts for a corpus with known *k*, now including the phase-two overhead: three steps per allocation gadget ([3, Lem. 6.7]) and two per erected atom ([3, Rem. 3.11]).

Obligation 11.7 (Differential testing against B3LL0WS). For a corpus of K0 programs, run the source under B3LL0WS and the compiled term under `f1r3comb-run` to quiescence and compare

observations. The observation **MUST** be the multiset of messages at names that are encodings of source names — the encoded observer class of [3, Thm. 7.9] — and **MUST NOT** include allocated leaves or static channels, which by [3, Lem. 7.6] no encoded context can name.

Obligation 11.8 (A against B). The corpus **MUST** be run under both presentations, unscoped: with [3, Def. 3.8]’s member for shapes with a process argument, B expresses the whole translation, and v0.3’s exclusion is withdrawn (Remark 2.2). The two readings of `consq` **MUST** additionally be compared atom by atom on every erected gate (Requirement 4.8).

Obligation 11.9 (Property tests). On generated K0 terms: both phases total; no binder in the target (Requirement 4.13); no drop under A; no constructor with a repeated or self-delivering channel; the family is one level deep (Requirement 4.7); leaf demand matches supply (Requirement 5.12); one spare per nested unit (Requirement 5.11); size bounds (Requirement 8.9); deploy-then-quietse terminates for terminating sources.

Obligation 11.10 (Address independence). [3, Lem. 7.8]: a unit run at σ and at τ prefix-incomparable are bisimilar. Tested by running the corpus at two unrelated root addresses and comparing observations. A failure here is a machine bug, not a compiler bug.

12 Errors and diagnostics

Code	Phase	Meaning
<code>comb-level</code>	1	A form above K0
<code>comb-unguarded-drop</code>	1	A drop of a name not bound by an enclosing input; warning under A, error under B
<code>comb-drop-quote</code>	1	A literal <code>*@Q</code>
<code>comb-open</code> , <code>comb-unforgeable</code>	1	Grounding survived, or an unforgeable name
<code>comb-occ-unclassified</code>	2	Internal: an occurrence matching no kind
<code>comb-arity</code>	4	A shape whose constructor arity exceeds the join bound (Requirement 9.2)
<code>comb-discipline</code>	4	The emitted image violates [3, Def. 6.10] (Requirement 5.14)
<code>comb-o5-address</code>	4	An address-derived channel feeds an (o5) chain (Requirement 5.20)
<code>comb-static-not-fresh</code>	deploy	A static channel does not exceed the largest name of $\llbracket P \rrbracket$ (Requirement 5.22)
<code>comb-address-too-small</code>	deploy	The root address does not (Requirement 5.19)
<code>comb-address-overlap</code>	deploy	A root address comparable with one in use (Requirement 5.23)
<code>comb-presentation-mismatch</code>	deploy	Artefact header disagrees
<code>comb-name-budget</code>	run	Name size or count exceeded
<code>comb-decode</code>	any	A source tag, or the drop tag under A, where a target tag was expected

13 Performance and portability

Requirement 13.1 (Complexity). Phase one **MUST** be linear in the source normal form. Phase two **MUST** be linear in the phase-one image, or $O(n \log n)$ under Requirement 5.16. Encoding **MUST** be linear and computed once per node. A machine step **MUST** be $O(1)$ expected in the size of the store, *given* the prefix sharing of Requirement 9.7; without it, hashing an address of depth d is $O(d)$ and a recursive program degrades quadratically, which **MUST** be a measured test and not an assumption.

Requirement 13.2 (Targets). Compile a 10^4 -node source term in under 10 ms; deploy 10^5 atoms in under 100 ms; sustain 10^6 machine steps per second free-running; and sustain 10^4 unfoldings of D_x without the step cost rising, which is the test that Requirement 9.7’s prefix sharing works. Budget figures, to be re-stated once measured.

Requirement 13.3 (Portability and determinism). Every crate **MUST** build for `wasm32-unknown-unknown` without `wasm-bindgen`, carry `forbid(unsafe_code)`, and take no external dependency. Given the same source, presentation, construction and root address, the compiled term, its encoding, its hash, the event stream and the final scene **MUST** be bit-identical across runs, hosts, core counts and WebAssembly versus native.

14 What the implementation asks of the papers

Seven items v0.3 raised are discharged by the revision: the permutation claim is now [3, Def. 6.10] and [3, Lem. 6.11]; phase-two freshness is [3, Lem. 6.14]; the storage constructor is in [3, Def. 3.8] with [3, Rem. 3.9] on the two readings; per-node gadget channels and the spare-leaf count are [3, Rem. 6.13]; the family’s finiteness is [3, Rem. 6.15]; and [3, Lem. 9.2] is weighted, with [3, Prop. 9.9] restated. Two items from v0.2 still stand. Three are new, and two of those are consequences of the repairs rather than gaps in them.

1. **ERECT does not satisfy the discipline it is now measured against** (Requirement 5.15). [3, Def. 6.8] emits $\text{cons}_A(t_{i_1}, \dots, t_{i_m}, f_A)$ at static t_i carrying leaves, so every atom mentioning two or more bound names gives a constructor with $m \geq 2$ address inputs at static channels, which [3, Def. 6.10] forbids. The distributor’s $\mathbf{d}(x, p_0, q_1)$ and the gate’s $\mathbf{s}(p_0, b, c)$ are both such atoms, so this is the normal case, not an edge. [3, Rem. 6.12] supplies the repair but is offered as “one way”. Proposed amendment: restate Definition 6.8 in the conforming form, or mark it explicitly as the undisciplined presentation to which Lemma 6.11 does not apply.
2. **Conforming requires two stages, and that is worth saying** (Requirement 5.16). [3, Rem. 6.12] asks that every join happen at an address-derived channel. An atom stands at a computed name only by being erected there ([3, Rem. 3.11]), so a conforming image necessarily has a single-input bootstrap at static channels, which erects a per-instance erector at address-derived channels, which erects the unit. The remark reads as though the discipline were a matter of routing; it is a matter of structure. Proposed amendment: say so, and note that fixed-length paths are what make leaf distinctness follow from injectivity rather than from the order the chains ran in.
3. **No freshness condition on the static channels** (Erratum 5.21). [3, Lem. 6.14] makes the leaves fresh against $\mathcal{N}^*(\llbracket P \rrbracket)$; nothing makes r , the gadget channels or the f_A fresh. An encoded context that names r can post an address of its choosing, defeating the hypotheses of both [3, Lem. 6.11] (prefix-incomparability) and [3, Lem. 6.14] (size). The same size argument fixes it. Proposed amendment: state the condition on the statics in §6.3 and extend [3, Lem. 7.6] to cover them.
4. **The enumeration order of occurrences is observable**. From v0.2, and still open. [3, Def. 6.3] says only “let $o_1, \dots, o_k$ enumerate the free occurrences”; the order decides which bound name is c_1 , hence which leaf, hence the bytes.
5. **Coincident occurrence kinds**. From v0.2, and now urgent: [3, Table 1]’s note is about occurrences, not kinds, and R of [3, §6.1] is exactly the coinciding case on which the old scheme failed. Proposed amendment: say that each occurrence has one kind, and say whether fusing two occurrences into one proxy is permitted — it is not, and no amount of per-instance allocation would repair it.

6. **The formation table’s m row is presentation-specific**, seconded from [4, Rem. 2.6]; and [3, §2]’s **source calculus is not** [5, Def. 2.6]’s (Obligation 3.2), the two agreeing exactly on guarded-drop terms.

15 Open questions

1. **Address growth, measured** (Hazard 9.6). Under phase two every loop descends, so this is now the common case rather than the pathological one. Does a real program need address compaction — reusing a retired subtree — and is that safe?
2. **Is the full family necessary?** [3, §12] asks it and [3, Rem. 6.15] narrows it to the shapes a program erects. With freshness now by size rather than by shape, the question is no longer load-bearing for correctness, only for the atom count and the lattice point.
3. **Receptivity** ([4, open item 5]). If the logic gains it, the **m/q** argument for A weakens, and with Remark 2.2 the balance on storage already leans the other way; what would not move is the injectivity of quotation.
4. **An optimising decoder.** The one development that would force B, and no longer blocked by anything in phase two (Remark 2.2).
5. **The matrix encoding.** A reduction graph as an upper-triangular matrix wants a bounded name universe; phase two makes the universe grow with every unfolding, which is a real constraint on that programme and worth stating before it is designed around.
6. **Composition with the π compiler, and a decompiler** as a round-trip canonicity check.

16 Delivery

M	Deliverable	Done when
M0	f1r3comb-term	Shape table, generated atoms and constructors, drop elimination, encoding, hash, decoder, multiset; tag and rigidity tests
M1	f1r3comb-ir , phase one	Binder, de Bruijn, extrusion, α -equivalence; checks; occurrence analysis; (o5) chains; canonicity property test; golden (ii)
M2	Phase two	Family computation, two-stage erection (Requirement 5.16), allocation, spares, fan-out; the [3, Def. 6.10] conformance pass and the (o5) address pass; golden (iii), the D_x run
M3	f1r3comb-space , f1r3comb-run	Matcher, generated rule table, address allocator, deploy, trace; conflation and freshness regressions
M4	f1r3comb-check	Golden (i), the W regression; the adversarial scheduler and Obligation 11.2; differential testing
M5	F1R3x , WASM	The five subcommands; wasm32 ; performance measured and Requirement 13.2 re-stated

The weight has shifted again, and in the right direction. Under v0.3 the differential test was the only evidence for phase two’s central invariant; with [3, Lem. 6.11] proved, the primary evidence is the conformance check of Requirement 5.14, which belongs to M2 and costs a dataflow pass. M4 returns to being a confidence measure — with the adversarial scheduler retained, because the lemma’s hypotheses are about the machine as much as the term.

17 Changes from v0.3

v0.3	v0.4
Obligation 5.13, permutation invariance unproved	Proved: [3, Def. 6.10] and [3, Lem. 6.11]. Replaced by Requirement 5.14, a mandatory conformance check, plus Requirement 5.15 — ERECT as stated does not conform — and Requirement 5.16, the construction that does
Remark 5.14, a scheme that narrows the obligation	Now [3, Rem. 6.12], and effectively mandatory rather than optional
Erratum 5.15, no freshness lemma; shape-based root address	[3, Lem. 6.14], by size. Requirement 5.19 is a deploy-time size test, materially simpler than the shape test v0.3 proposed
—	New: Erratum 5.21, the static channels have no freshness condition, which is the last route by which an encoded context could reach the apparatus
—	New: Requirement 5.20, the second clause of [3, Lem. 6.14] as a compiler check, scoped to post-erection flows
Erratum 4.9, no member for q ; phase two A-only	[3, Def. 3.8] has the member. v0.3 was wrong in the interesting direction: B builds it more directly (Remark 2.2). The fourth reason for A is withdrawn; Obligation 11.8 is unscoped
Requirement 4.6, family finiteness argued here	Now [3, Rem. 6.15]; the requirement cites it and adds Requirement 4.9, gate payloads reaching further shapes
Errata 5.9 and Requirement 5.11, per-node channels and spares	Now [3, Rem. 6.13]; kept as requirements citing it
Requirement 10.3, the weighted count argued here	Now [3, Lem. 9.2] with [3, Prop. 9.9] restated
Nine items asked of the papers	Six, of which seven were discharged, two stand from v0.2 and three are new

References

- [1] F1R3FLY.io. *Bytecode or normal form: what the executive should execute*. [publications/campf1r3/bytecode-or-normal-form](#), 2026.
- [2] F1R3FLY.io. *CampF1R3: an in-memory tuple space*. [publications/campf1r3/campf1r3-spec](#), 2026.
- [3] L. G. Meredith and M. Stay. *Name-free combinators for the rho calculus: a repair, a linear encoding, and the one operation that was missing*. Draft 3, [publications/rho-combinators](#), September 2026.
- [4] F1R3FLY.io. *A spatial-behavioural logic for the rho combinators*. Draft 2, revision 1, [publications/rho-combinators](#), 22 September 2026.
- [5] F1R3FLY.io. *K1ndl1ng: a parser and a normaliser for the kernel of rholang*. Specification v0.2, [publications/campf1r3/k1ndl1ng-spec](#), 2026.
- [6] L. G. Meredith and M. Radestock. *A reflective higher-order calculus*. ENTCS 141(5):49–67, 2005.
- [7] L. G. Meredith. *Namespaces without a name server*. [publications/pi2rhobugfix](#), F1R3FLY.io, 2026.
- [8] L. G. Meredith and M. Stay. *Name-free combinators for concurrency*. Working draft, rho4u.
- [9] N. Yoshida. *Minimality and separation results on asynchronous mobile processes*. Theoretical Computer Science 274(1–2):231–276, 2002.